

Distributed TTT with Java RMI

Analisi dei requisiti

Per questo progetto era richiesta la creazione di un sistema distribuito allo scopo di creare sessioni per giocare a Tic-Tac-Toe.

Architettura Client-Server tramite java RMI:

Il sistema si basa su infrastruttura in cui un Server funge da host, gestendo molteplici richieste da parte dei Client Distribuiti sulla rete. Il server, tramite un registro chiamato RMI Registry, è in grado di pubblicare le interfacce remote, gettando le basi per la creazione di collegamenti su cui si baseranno i Client.

Funzionalità di base per il giocatore:

- **Creazione:** Un giocatore deve poter creare una nuova sessione di gioco assegnando un nome univoco e mettendosi in attesa passiva di un avversario.
- **Accesso:** Un giocatore deve poter cercare una stanza esistente tramite il suo nome e unirsi alla partita per farla iniziare.
- **Gioco:** Entrambi i giocatori devono essere in grado di eseguire le proprie mosse al turno corretto, finché non si arriva a una condizione parità o di vittoria di uno dei due

Comunicazione ad eventi:

Si preferisce generalmente utilizzare sistemi a callback remoti, data un'azione, una reazione, che un sistema ciclico basato sul polling continuo, perché in questo caso le performance ne andrebbero a risentire. Il client espone un listener remoto, che invocherà il server attivamente solamente quando avviene un cambiamento di stato. Nel nostro dominio specifico questo è dato ad esempio da una mossa effettuata o dall'entrata di un giocatore in una partita specifica.

Utilizzo degli Oggetti Stub:

Per rendere un oggetto accessibile, ad esempio una sessione di gioco, esso deve essere esportato su una porta specifica dal server. Questo processo genera uno "Stub" a lato del client, nella pratica un proxy in locale che maschera completamente la complessità della comunicazione di rete, permettendo al client di chiamare i metodi della partita come se l'oggetto fosse in memoria locale.

Architettura

Il progetto è basato sulla programmazione distribuita, implementata tramite la tecnologia Java RMI (Remote Method Invocation). Inoltre, l'architettura segue il modello Client-Server.

Il sistema è composto dai seguenti nodi:

- **Server (RunServerTTT):** configura l'hostname tramite `System.setProperty("java.rmi.server.hostname", hostIp)`; e avvia

il registro RMI sulla porta 1099. Se il registro è già attivo, lo recupera per evitare l'eccezione `ExportException`. Successivamente, istanzia il `GameManager` e lo esporta tramite il comando `UnicastRemoteObject.exportObject(manager, 0)` ; , pubblicandolo nel registro con il nome "GameManager"

- Client (RunClientTTT): ogni client localizza il `GameManager` nel registro. Inoltre, ognuno di essi istanzia la classe `RemotePlayerListener`, la esporta (lproxy) e la passa al server. In questo modo, il server ottiene un riferimento per comunicare con il client

Abbiamo definito tre interfacce che estendono `java.rmi.Remote`. Questo perché per permettere la comunicazione tra macchine diverse, ci siamo basati sul meccanismo dei proxy fornito da RMI. Il funzionamento è semplice: quando il client effettua la lookup sul registro per ottenere il `GameManager`, non riceve una copia dell'oggetto server, ma scarica uno "Stub", ossia un proxy. Lo Stub, che implementa a sua volta l'interfaccia `Remote`, si occupa di tutto il lavoro di rete: impacchetta i parametri, li manda al server, fa eseguire la logica da remoto e ci restituisce il risultato. Chiaramente, questa comunicazione che avviene attraverso la rete è soggetta a ritardi o problemi improvvisi come disconnessioni: per questo motivo, Java RMI impone che ogni metodo dell'interfaccia `Remote` dichiari `throws RemoteException`. Di conseguenza, nel nostro codice abbiamo dovuto gestire eventuali fallimenti di rete tramite blocchi try-catch.

Il **GameManager** è colui che si occupa di implementare lato server le partite. Permette agli utenti di unirsi in match formati da due giocatori. Viene usata la seguente struttura dati per memorizzare le partite in attesa di un secondo giocatore.

```
Map<String, GameSession> pendingGames
```

I metodi che presenta sono due:

- `createGame(String gameName, RemotePlayerListener user1)`: è il metodo che si occupa di creare una nuova stanza di gioco. Ogni stanza avrà un determinato nome che viene inserito da tastiera da parte dell'utente. Il metodo verifica se la mappa `pendingGames` contiene già il nome inserito, e in caso negativo istanzia un nuovo `GameSessionImpl`. Questo oggetto viene poi esportato tramite `UnicastRemoteObject` per ottenerne lo Stub, che viene inserito nella mappa e restituito al Client (impersonificato dal giocatore 1)
- `joinGame(String gameName, RemotePlayerListener user2)`: se la stanza esiste nella mappa `pendingGames`, recupera lo stub della partita (che viene immediatamente rimosso dalla mappa) e lo restituisce al Client (impersonificato dal giocatore 2), chiamando allo stesso tempo il metodo `gameStub.addSecondPlayer(user2)`

GameSession: è l'implementazione lato server della singola partita di Tic-Tac-Toe. Gestisce lo stato della griglia, che abbiamo deciso di modellare come un array monodimensionale `String[] board` di 9 elementi (inizializzati a stringa vuota ""). La classe si occupa della gestione dello stato dei turni e della logica di vittoria o pareggio. Inoltre, manda messaggi ai client per notificare loro il cambiamento di stato. Il metodo principale:

- `makeMove(int pos)`: è il metodo che si occupa di aggiornare lo stato del gioco dopo l'esecuzione di una mossa. Come parametro, riceve un numero da 0 a 8 che rappresenta la cella cliccata. Determina il simbolo da inserire (X o O a seconda di chi

è il turno), aggiorna la griglia (l'array) e incrementa la variabile moveCount, che detiene il numero di mosse attualmente fatte da entrambi i giocatori. Lo stesso metodo si avvale di un metodo privato per controllare se il gioco è finito. Itera su una matrice formata da combinazioni vincenti (per esempio, fare tris sulla prima riga in alto equivale a fare {0,1,2}, mentre fare tris sulla prima colonna a sinistra equivale a {0, 3, 6}). Se tutte e tre le posizioni contengono lo stesso simbolo, viene rilevato il tris. Di conseguenza, il metodo makeMove invoca direttamente la callback onGameOver sui listener dei due giocatori. Nel caso in cui non sia rilevata vittoria, il metodo controlla se il numero di mosse fatte corrisponde a 9: in tal caso, la partita è terminata con un pareggio. Altrimenti, il gioco continua con l'inversione del turno e l'invocazione di onGameUpdate su entrambi i listener

RemotePlayerListener è una classe la cui implementazione risiede sul client e riceve aggiornamenti passivi. È un oggetto esportato via RMI al server, che lo usa per invocare passivamente aggiornamenti sul client. Inoltre, tale classe è il tramite tra la GUI e il server. Infatti, mantiene un riferimento alla GUI e si occupa di richiamare invokeLater per effettuare gli aggiornamenti grafici

Gestione della sincronizzazione:

In Java RMI, ogni invocazione proveniente da un client viene gestita dal server assegnando un thread separato. In mancanza di sincronizzazione, quindi, il sistema sarebbe esposto a Race Conditions, condizioni di interferenza (interazione non voluta) in cui il risultato del programma cambia in modo imprevedibile. Per risolvere il problema, abbiamo utilizzato la keyword synchronized sui metodi createGame e joinGame in GameManagerImpl. Il motivo è che se due giocatori provassero a invocare lo stesso metodo con lo stesso nome di partita nello stesso istante, entrambi i thread potrebbero superare il controllo if (pendingGames.containsKey(gameName)) prima che uno dei due sia riuscito ad inserire il nome nella mappa. Rendendo il metodo sincronizzato, le richieste vengono processate in mutua esclusione. La keyword synchronized l'abbiamo usata anche per i metodi addSecondPlayer e makeMove in GameSessionImpl, in modo da garantire mutua esclusione e atomicità nell'inserimento di una mossa.

Sistema di GUI:

Per rendere più piacevole l'interazione tra il gioco e l'utente, è stata progettata una semplice GUI in Java Swing, che copre interamente la parte di gioco effettiva, a seguito della creazione o dell'entrata in una partita (operazioni invece che necessitano la linea di comando).

Componenti fondamentali:

- Nome della partita
- Nome dei player
- Stato del player: in attesa nel caso di turno avversario o quello in cui si sia creato una partita e nessun giocatore vi abbia ancora partecipato. Altrimenti mostra che è il nostro turno di eseguire una mossa
- Info down, provvede informazioni sullo stato del gioco, come:
 - In attesa del server
 - Torno del giocatore
 - Esito della partita(vittoria di giocatore, parità)

- Griglia: È composta da actionlistener, cliccabili solamente durante il turno del giocatore e se la suddetta casella è libera. Quando un clic viene eseguito su una casella valida allora verrà richiamato: `session.makeMove(cellIndex)`; Metodo che permetterà alla sessione di registrare la mossa appena effettuata, rendendone così visibili gli effetti anche all'altro giocatore



Immagine della creazione di una stanza in cui il player 2 non è ancora entrato a far parte del gioco.



Immagine dello svolgimento di una partita mostrando entrambe le istanze dei client giocatori.

Far girare il programma

le informazioni per far girare correttamente il programma sono presenti nel README della repository, disponibile in lingua italiana e lingua inglese. Abbiamo strutturato il progetto in modo tale da permettere anche partite su computer diversi ma connessi alla stessa rete locale, grazie all'utilizzo della stessa rete IPv4 presentata dal server.